

ConnectCoin

A Peer-to-Peer Cryptocurrency for TLS-Based Rewards

Papaulo

September 2026

Abstract

ConnectCoin is a Bitcoin-derived blockchain that associates transaction outputs with cryptographically verifiable TLS interactions. A participant creates a Pay-to-Connect output specifying a domain, a reward, a connection-work target, and permitted signature schemes. Another participant prepares a spending transaction and obtains a server-authenticated TLS handshake transcript bound to that transaction. Nodes verify the submitted evidence without contacting the domain. A hash target selects which transcripts qualify for payment, allowing fewer proofs to be recorded than the number of candidate transcripts examined. The work hash covers the authenticated handshake prefix, with the server signature retained as evidence. RandomX proof-of-work separately secures block ordering. A declining issuance schedule and a subsidy reduction proportional to transaction weight establish the monetary and block-space incentives. This paper describes the construction and its economic policy.

1 Introduction

A digital payment can reward a network interaction only if the parties can agree on evidence that the interaction occurred. A recipient's own report is not sufficient evidence, while asking every blockchain node to repeat the interaction would make validation depend on network access, changing server state, and local timing. The objective is therefore to separate obtaining an external response from verifying the evidence used to claim a payment.

TLS provides a useful starting point: a server authenticates part of its handshake using a certificate-backed signing key. ConnectCoin makes a restricted form of that authenticated transcript a transaction-spending condition. A claim contains the messages and certificates needed for deterministic verification, and the server's response is bound to the transaction that receives the reward. Payment does not require the funder to approve the claimant after seeing the result.

The proposal has two layers. *Proof-of-Connect* defines the evidence and its connection-work condition. *Pay-to-Connect* attaches that condition to an unspent transaction output. These are distinct from block production: RandomX proof-of-work orders transactions, whereas P2C proofs spend rewards already supplied by other participants. A successful P2C claim does not itself mint currency or create a block.

The intended result is a market for narrowly defined, verifiable interactions, not a universal oracle for Internet activity. A transcript does not establish that a human visited a website, that content was delivered, or that the server is independent of the claimant.

2 Network Specifications

ConnectCoin reuses the Bitcoin Core codebase and the UTXO model, in which inputs consume previous outputs and new outputs define subsequent spending rights. Transactions are ordered in blocks, and nodes select the valid chain with the greatest accumulated proof-of-work [1]. Changes to transaction encoding, proof-of-work, and network parameters make ConnectCoin

incompatible with the Bitcoin network. Despite the fact that partial implementation was reused, there will be no compatibility with Bitcoin balances, wallets, or historical blocks.

Block production and capacity

Block proof-of-work uses RandomX V2 over the serialized 80-byte header. RandomX is designed around general-purpose CPU execution and substantial memory requirements [3]. By favoring general-purpose CPUs over GPUs and seeking to minimize the efficiency advantage of application-specific integrated circuits (ASICs), this design aims to support home mining on widely available hardware and encourage a more decentralized distribution of mining power.

The configured target interval is 10 seconds. The ordinary difficulty adjustment period is 8,640 blocks, corresponding to one day at that interval. Difficulty adjustment must not be confused with the RandomX key schedule: the latter uses 12,288-block epochs with a 64-block lag and a network-specific bootstrap key.

Parameter	Public test-network setting
Target block interval	10 seconds
Ordinary difficulty adjustment	8,640 blocks
Maximum block weight	50,000,000 weight units
Coinbase maturity	100 blocks
Initial recurring subsidy	15 CONN per block
Halving interval	3,000,000 blocks
Smallest monetary unit	10^{-10} CONN, one connect

Weight follows the distinction between transaction base data and witness data used by segregated witness [4]. If b is the stripped size and t is the total serialized size, then

$$w = 3b + t = 4b + (t - b). \quad (1)$$

Consequently, 50 million weight units are not generally 50 million bytes of ordinary transaction data. Depending on the proportion of witness data, serialized blocks can approach 50 MB within this weight limit.

Typed outputs

Instead of general transaction-output scripts, the current protocol recognizes two output types. Type 1 contains a 32-byte x-only secp256k1 public key and is spent with a 64-byte BIP340 Schnorr signature [5]. Its transaction signature digest follows the BIP341-style default signing construction [6]. Type 2 is the Pay-to-Connect output defined in Section 4. Unknown output types are rejected. This restricted format intentionally does not implement arbitrary Bitcoin Script spending conditions.

3 Proof-of-Connect

Binding a response to a claim

Let C be a prepared claim transaction, with the reward destination and all non-witness fields already fixed, and let i be its P2C input index. The connection challenge is

$$c = \text{TaggedHash}_{\text{claim}}(\text{txid}(C) \parallel \text{LE32}(i)). \quad (2)$$

The tag is **ConnectCoin/P2C/claim/v1**. Transaction-ID bytes use the protocol serialization, and the input index is a four-byte little-endian integer. Tagged hashing uses the domain-separation construction $\text{SHA256}(\text{SHA256}(\text{tag}) \parallel \text{SHA256}(\text{tag}) \parallel m)$ [5].

The claimant puts the exact 32 bytes of c in **ClientHello.random** and sets the requested domain in the server-name extension. A new connection attempt can retain this challenge while generating a fresh key share. The server need not run a ConnectCoin-specific application: a compatible TLS implementation authenticates the handshake containing these client fields.

The transaction ID excludes witness data, avoiding a circular dependency between the claim and its proof. It nevertheless commits to the consumed outputs and the payout transaction. Changing a recipient, fee, or other non-witness field changes the challenge and requires new server-authenticated evidence.

Authenticated evidence

Proof version 2 contains the raw handshake messages

$$\tau = CH \parallel SH \parallel EE \parallel CRT \parallel CV, \quad (3)$$

Here CH is **ClientHello**, SH is **ServerHello**, EE is **EncryptedExtensions**, CRT is **Certificate**, and CV is **CertificateVerify**. Each includes its TLS handshake header. The proof is the byte 0x02 followed by these messages, in this order. It excludes TLS record framing, session secrets, **Finished**, and application data [2].

The TLS 1.3 server signature authenticates the handshake prefix through **Certificate**, using the TLS server context and the public key from the leaf certificate [7]. A validator checks that the certificate chain is accepted for the specified domain, that the expected challenge is present, and that the server signature verifies. The strict inference is therefore an authenticated transcript under a key authorized for that domain, not proof that every phase of a TLS session completed.

ConnectCoin accepts a deliberately restricted TLS 1.3 profile: AES-128-GCM or ChaCha20-Poly1305 with SHA-256, X25519 or secp256r1 key shares, and ECDSA P-256/SHA-256 or either supported RSA-PSS/SHA-256 authentication scheme. The selected signature scheme must be offered in the **ClientHello** and permitted by the output's mask. RSA keys must contain at least 2,048 bits; RSA-PSS uses MGF1-SHA-256 and a 32-byte salt. RSA encryption keys and restricted RSA-PSS keys are distinguished and checked against the selected scheme. Resumption, early data, and **HelloRetryRequest** are excluded. This reduces the set of states that consensus must interpret, at the cost of compatibility with some servers. The exact versioned profile, rather than the entirety of any TLS standard, determines valid P2C evidence. Later TLS specification revisions do not automatically change consensus [8].

Certificate roots form an immutable, versioned consensus bundle. Validators use that bundle rather than their operating system's changing trust store. The validity dates of supplied certificates are evaluated at the previous block's median time past when validating a block, and at the current tip's median time past for mempool admission. No DNS lookup or live certificate-status request is part of this verification.

Connection-work selection

Define the authenticated prefix $M = CH \parallel SH \parallel EE \parallel CRT$. The candidate work hash is

$$H = \text{TaggedHash}_{\text{work}}(M), \quad H \leq T, \quad (4)$$

where the tag is **ConnectCoin/P2C/work/v2** and T is the output's 256-bit target. These four messages include their handshake headers. The entire **CertificateVerify** message and the

proof-version byte are excluded from this hash. The digest is interpreted using ConnectCoin’s little-endian 256-bit integer representation. The target comparison is inclusive.

The server’s CertificateVerify signature authenticates M and is required for a valid proof.

For independent, uniformly distributed candidate hashes, the success probability and expected number of candidate evaluations are

$$p(T) = \frac{T+1}{2^{256}}, \quad \mathbb{E}[K] = \frac{1}{p(T)}. \quad (5)$$

The probability of seeing no qualifying candidate after n evaluations is $(1-p)^n$. For example, $T = 2^{246} - 1$ accepts hashes with ten leading zero bits in their numeric representation and gives $p = 1/1024$.

4 Pay-to-Connect

Funding and redemption

A Pay-to-Connect output has a value V and payload

$$(d, T, r, m), \quad (6)$$

where d is a lower-case ASCII domain satisfying the protocol’s label grammar, T is its connection-work target, r identifies the trusted-root bundle, and m is a mandatory one-byte `signature_algorithms_mask`. Domain length is encoded in one byte, the target occupies 32 bytes, and the root version occupies four. With the eight-byte amount and one-byte output type, the complete serialized output occupies $47 + |d|$ bytes.

Mask value	CertificateVerify scheme	TLS identifier
1	<code>ecdsa_secp256r1_sha256</code>	<code>0x0403</code>
2	<code>rsa_pss_rsae_sha256</code>	<code>0x0804</code>
4	<code>rsa_pss_pss_sha256</code>	<code>0x0809</code>

For example, mask 7 permits all three schemes; mask 6 permits both RSA schemes. Zero and reserved bits are invalid. This mask restricts server authentication, not the issuer signatures along the certificate chain. The RPC default is 7. When funding through the graphical wallet, a capability probe offers only the two RSA schemes during the three-second confirmation delay. A full, authenticated handshake completed before the deadline selects 6; failure or timeout retains 7. The choice freezes before confirmation is enabled. This advisory probe uses local time and the selected endpoint’s current capability; it cannot guarantee future acceptance under chain-time validation. One advantage of this preference is the lower computational cost of RSA signature verification.

Anyone can fund this output, and anyone satisfying its spending condition can claim it. There is no creator signature required at redemption and no creator-only refund branch. In particular, the current type does not contain a deadline or a mechanism to recover funds if the domain becomes permanently unusable. Funding a bounty is a commitment to its encoded conditions.

A simple claim spends one bounty and creates an ordinary output paying $V - f$ to the claimant, where f is the transaction fee.

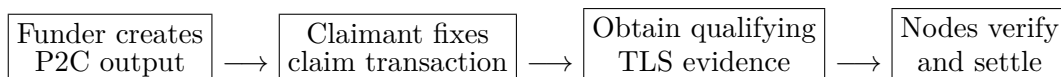


Figure 1: Only the claimant obtains the external response. Validators use the evidence carried by the claim transaction.

For each P2C input, validators require an existing, spendable output; the correct proof version and message structure; the transaction-bound challenge; the matching domain and accepted certificate path; a valid, mask-permitted server signature; and a work hash satisfying the output target. These checks supplement ordinary transaction, block, and double-spend validation. Each P2C input has an empty scriptSig and exactly one proof witness element. The full proof is limited to 64 KiB (65,536 bytes). The Certificate message is limited to 48 KiB, containing at most eight certificates of at most 16 KiB each; CertificateVerify has its own 8 KiB limit.

Competition and reuse

Each bounty is a single-use UTXO. Multiple claimants may search concurrently, but only one conflicting spend can survive in the selected chain. Producing a valid proof does not guarantee payment: another claim can be confirmed first, and a reorganization can change the accepted history. A third party can relay a completed claim but cannot redirect its outputs without changing the challenge in Equation (2).

The input-index binding also prevents simply copying one proof to several inputs of the same transaction. A funding transaction can create many independent rewards for the same domain, but each output remains a separate claim opportunity. Splitting a fixed budget can provide more independently settled outcomes while increasing funding and claim overhead; it does not create additional value.

There is no special P2C confirmation-age rule in consensus for ordinary outputs beyond the usual spending and lock conditions. The automatic wallet waits for confirmed bounties as a local policy. Coinbase outputs separately retain their 100-block maturity. Search limits, domain scheduling, and decisions to abandon an unproductive bounty do not invalidate otherwise valid claims.

Automatic claim selection

The wallet first discovers eligible outputs using an in-memory P2C catalog initialized from the UTXO set. Later updates apply newly connected blocks and undo reorganized blocks; unavailable history can require rebuilding the catalog. The discovery window is configurable: by default it covers the latest 600 blocks, including the tip, approximately 100 minutes at the target interval. Unlimited history includes all confirmed ages (`recent_blocks=0`). The wallet warns that large windows can include old, unproductive bounties. This is a soft discovery window, not on-chain expiry: crossing its boundary does not cancel an in-flight attempt.

The wallet refreshes the candidate set every five seconds and selects a domain for each new connection. Assignments alternate a fair turn among eligible domains and a turn for the domain with the highest estimated reward rate. Thus rewards influence cross-domain priority without making it absolute. The economic score uses the best eligible bounty within a domain, not the sum of its advertised rewards. There is no fixed per-domain connection quota.

For each pair of domain and signature mask, the wallet retains the last 100 completed TCP/TLS attempts. Write s for captures reaching CertificateVerify and t for the sum of attempt durations in seconds. Its smoothed capture rate is

$$q = \frac{0.1 + s}{0.02 + t}. \quad (7)$$

Before any samples, $q = 5$ captures per second of connection effort. Failures and timeouts contribute time without success; locally canceled incomplete attempts are excluded. DNS resolution, scheduling waits, and later certificate verification are not timed here. Reaching CertificateVerify is a capture success, not necessarily a valid or winning claim. Concurrent attempt durations are added independently, so this is not wall-clock throughput.

With net bounty value $V - f$ measured in connects, the estimated reward rate is

$$G = (V - f) \frac{T + 1}{2^{256}} q. \quad (8)$$

Automatic selection requires $G \geq 1,000$ connects per second of connection effort. This is a monetary threshold, not 1,000 connections per second. This estimate does not deduct operating costs or account for competing claims.

The wallet stops starting new attempts for a bounty once its number of successful captures through CertificateVerify exceeds $2/p(T)$, whether or not those captures yield valid, winning claims. Failed connections and incomplete local cancellations do not consume this count. With sequential attempts, the cutoff is $N = \lfloor 2/p(T) \rfloor + 1$ captures: five when $p = 1/2$. Concurrent attempts already underway may finish and submit proofs after the cutoff. Counters remain local to the loaded wallet, surviving stop/start until their bounty leaves discovery and pending work drains; they reset on wallet reload. For independent uniform candidates with small p , no win in approximately $2/p$ trials has probability near $e^{-2} \simeq 13.5\%$. This is a resource-management heuristic, not evidence that the server cheated.

Automatic searching is opt-in. The graphical wallet proposes a limit of 100 connection starts per second and independently defaults to 100 simultaneous connections. Higher settings and an unlimited rate remain available, with a prominent warning. HTTPS remains disabled until the user starts it and accepts the confirmation. The rate is shared by the workers of that wallet, not applied separately to each domain. Both the block miner and automatic P2C claims use a receiving destination from the selected wallet by default, while allowing an explicit supported address. A completed proof is bound to the destination fixed in its claim; it cannot be retargeted afterward. All selection rules in this subsection are wallet policy, not additional consensus conditions for manually submitted claims.

5 Economic Policy

Units, allocation, and halvings

One CONN contains 10^{10} connects, the atomic accounting unit. The intended nominal supply budget is 100 million CONN. The proposed launch allocation is:

- **Development Fund: 5% (5,000,000 CONN).** Restricted to hosting websites and rewarding independent developers of official ConnectCoin projects, including ConnectCoin Core, and high-quality unofficial projects that contribute substantial value, such as alternative wallets and block explorers. We commit to publicly explaining all expenditures from the Development Fund.
- **Developer's Fund: 5% (5,000,000 CONN).** The founder's personal allocation, intended to cover personal living expenses and provide an incentive for sustained work on ConnectCoin.

- **Ordinary mining: 90% (90,000,000 CONN).** The nominal recurring subsidy budget, subject to integer rounding and the weight-dependent reduction described below.

The Development Fund and the Developer’s Fund are issued together in the genesis block’s coinbase transaction, totaling 10,000,000 CONN.

We commit to keeping cumulative withdrawals from each fund within an allowance that increases by **1,000,000 CONN each year**. For each fund separately, this annual increment is 1% of the nominal total distribution, or 20% of that fund’s initial 5,000,000 CONN allocation. It defines a maximum, not a minimum or a withdrawal target. The annual increment is not recalculated as 20% of the remaining balance.

The initial 1,000,000 CONN allowance for each fund becomes available on the mainnet launch date, with another 1,000,000 CONN added on each calendar anniversary. Any unused withdrawal allowance carries forward, increasing the amount available for withdrawal in later annual periods. For example, if no withdrawals are made from a fund during the first period, up to 2,000,000 CONN may be withdrawn from that fund during the second period. Each fund retains its own cumulative allowance and purpose. This schedule authorizes withdrawals only from available balances; it does not create new allocations after the original funds have been used.

Let $h \geq 1$ be the height of an ordinary block, $L = 3,000,000$ the halving interval, and $S(h)$ the scheduled subsidy measured in connects. Then

$$S(h) = \left\lfloor \frac{15 \cdot 10^{10}}{2^{\lfloor h/L \rfloor}} \right\rfloor. \quad (9)$$

The initial ordinary subsidy is 15 CONN. Halving occurs by block height, not by calendar date: L blocks at the target interval take about 347.22 days. “Approximately annual” is therefore a description of the target schedule, not a promise of an exact year between halvings.

With a 10 million CONN initial allocation and every subsequent subsidy fully claimed, the exact integer schedule gives

$$10,000,000 + \frac{1}{10^{10}} \sum_{h=1}^{\infty} S(h) = 99,999,984.9955 \text{ CONN}. \quad (10)$$

The difference from 100 million follows from integer rounding and the genesis occupying height zero rather than receiving an additional ordinary subsidy. Actual issuance can be lower because of weight reductions or unclaimed subsidies. The separate 100 million CONN amount-range bound is a validation sanity limit, not itself the issuance formula.

Block-space incentive

A miner earns the scheduled subsidy for a block containing only its coinbase. Including other transactions reduces the available subsidy in proportion to their weight. Let $W = 50,000,000$ weight units denote the *consensus-enforced maximum total block weight*, and let w be the sum of all *non-coinbase transaction weights* in the block. For a valid block, $0 \leq w < W$. The reduction in connects is

$$P(h, w) = \left\lfloor \frac{S(h)w}{10W} \right\rfloor. \quad (11)$$

If F is the sum of transaction fees, the maximum total coinbase output is

$$R_{\max}(h, w) = F + S(h) - P(h, w). \quad (12)$$

The header and coinbase count toward the block-weight limit but not toward w . A block containing only its coinbase therefore incurs no reduction, while increasing non-coinbase weight brings the reduction toward 10% of the scheduled subsidy.

At $w = W/2$, the subsidy reduction is 5%, apart from rounding. Transaction fees can make total coinbase revenue exceed the base subsidy even when the subsidy itself has been reduced.

The withheld amount is never created. It is not transferred to a treasury, burn address, or other participants, and it is not recovered by the issuance schedule later. P2C claims transfer existing rewards; they are not another source of monetary emission.

For a package that fits within the block-weight limit, with additional weight Δw , the immediate marginal subsidy cost is $P(h, w + \Delta w) - P(h, w)$. A fee-revenue-oriented miner has an incentive to include it when its fees exceed that cost. The native block assembler uses this comparison with locally modified package fees. This is selection policy, not a consensus requirement that miners maximize revenue. Arithmetic is performed with exact integer rounding and overflow-safe intermediates.

Relay fees

One kilovirtualbyte corresponds to 4,000 weight units. The default relay fee rate is the next block's subsidy cost for that weight, plus a margin of 1,000 connects per kilovirtualbyte:

$$r(h) = \left\lfloor \frac{4,000 S(h)}{10W} \right\rfloor + 1,000 \quad \text{connects/kvB.} \quad (13)$$

Initially this is 1,201,000 connects/kvB, or 0.0001201 CONN/kvB; after the first halving it is 601,000 connects/kvB. The fixed margin remains when the subsidy-dependent term falls. Mempool conditions and local configuration can require a different rate. A node's relay policy is not an immutable minimum-fee rule for consensus-valid blocks.

When fee estimation is unavailable and no nonzero fallback rate is configured, automatically selected wallet fees use the greatest of the next block's economic mining floor, the current mempool minimum, and applicable wallet/relay minimums. Automatic P2C claims reserve a fee budget before fixing the transaction challenge. Their budget accounts for the maximum permitted proof size; unused budget remains a fee rather than being returned by changing an already authenticated payout. Fees therefore also affect eligibility in Equation (8).

6 Conclusion

We have presented a system for funding TLS-based interactions through open, predefined spending conditions. We began with the UTXO model, so that every bounty represents funds already committed and can be redeemed only once. We then bound each proof to a particular transaction and input, fixing the recipient before the search begins and preventing the evidence from being reused for a different input or payout. Compatible servers provide the required authentication through TLS without running ConnectCoin-specific software.

With these conditions in place, claimants can compete independently for the same reward, while the selected chain resolves conflicting spends. The funder does not need to select a claimant or approve the payment afterward, and transaction fees can be taken from the bounty itself. Participants choose which rewards to pursue and how much effort to commit, without making those choices additional consensus requirements.

Finally, the block-space incentive gives transaction inclusion an explicit economic cost: miners weigh fee revenue against the subsidy forgone by including additional transaction weight. Funding, claiming, and block production remain distinct activities, coordinated through shared transaction rules rather than individual arrangements between participants.

References

- [1] S. Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. 2008. <https://bitcoin.org/bitcoin.pdf>.
- [2] ConnectCoin contributors. *ConnectCoin Core*, revision 783840b236, September 2026. Protocol and monetary rules: `src/consensus/p2c.cpp`, `src/consensus/p2c_x509.cpp`, `src/kernel/chainparams.cpp`, `src/validation.cpp`, `src/wallet/p2c_worker_impl.cpp`, `doc/typed-outputs.md`, and `doc/p2c-wallet.md`. Source tree and protocol documentation.
- [3] tevador and RandomX contributors. *RandomX: Proof of Work Algorithm Based on Random Code Execution*. Design and reference implementation. <https://github.com/tevador/RandomX>.
- [4] E. Lombrozo, J. Lau, and P. Wuille. *Segregated Witness (Consensus Layer)*. BIP 141, 2015. <https://github.com/bitcoin/bips/blob/master/bip-0141.mediawiki>.
- [5] P. Wuille, J. Nick, and T. Ruffing. *Schnorr Signatures for secp256k1*. BIP 340, 2020. <https://github.com/bitcoin/bips/blob/master/bip-0340.mediawiki>.
- [6] P. Wuille, J. Nick, and A. J. Towns. *Taproot: SegWit Version 1 Spending Rules*. BIP 341, 2020. <https://github.com/bitcoin/bips/blob/master/bip-0341.mediawiki>.
- [7] E. Rescorla. *The Transport Layer Security (TLS) Protocol Version 1.3*. RFC 8446, August 2018. <https://www.rfc-editor.org/rfc/rfc8446>.
- [8] E. Rescorla. *The Transport Layer Security (TLS) Protocol Version 1.3*. RFC 9846, 2026. Update superseding RFC 8446. <https://www.rfc-editor.org/rfc/rfc9846>.